

Data_Architecture_Standard

Technical Standard

保险公司 技术开发部
内部文档 · 仅供授权人员查阅

Contents

1. Overview
2. Data Architecture Principles
3. Data Layering
4. Data Modeling Standards
 - 4.1 Layered Modeling
 - 4.2 Core Model Example - Policy Fact Table
5. Storage Selection
6. Data Pipeline Standards
 - 6.1 Real-time Pipeline
 - 6.2 Offline Pipeline
7. Data Governance
8. Regulatory Reporting

Data Architecture Standard

1. Overview

This document defines the overall data architecture standards for the insurance technology platform, including data layering, data modeling, storage selection, data flow, and data governance. References include Alibaba data methodology and Ant Group data governance practices.

2. Data Architecture Principles

1. **Unified Standards:** All data follows consistent naming, type, and encoding standards
2. **Layered Decoupling:** Data is organized as ODS → DWD → DWS → ADS, each layer with clear responsibilities
3. **Data Traceability:** Every data pipeline must support end-to-end traceability
4. **Compute Near Data:** Push computation as close to the storage layer as possible
5. **Least Privilege:** Data access follows need-to-know, minimum-sufficient principles

3. Data Layering

Layer	Description	Storage	Retention
ODS	Raw data ingestion unchanged	HDFS / Kafka	30 days
DWD	Cleaned detail, facts + dimensions	Hive / Iceberg	Per regulatory
DWS	Lightly summarized, subject-wide tables	ClickHouse	Per regulatory
ADS	Application-specific data marts	MySQL / ClickHouse	Per business
DIM	Common dimension data	MySQL / Redis	Long-term
STG	Temporary intermediate results	HDFS / OSS	Cleanup T+7

4. Data Modeling Standards

4.1 Layered Modeling

- **ODS:** 1:1 mapping from source systems, no cleaning
- **DWD:** 3NF or Data Vault, standardized fields, detail-level retained
- **DWS:** Subject-organized (Underwriting, Claims, Customer), light aggregation
- **ADS:** Application-specific (regulatory reports, business analytics, risk models)

4.2 Core Model Example - Policy Fact Table

```
CREATE TABLE dwd_policy_fact (
  policy_id STRING,
  application_id STRING,
  product_code STRING,
  channel_code STRING,
  applicant_id STRING,
  insured_id STRING,
  premium_amount DECIMAL(18,2),
  sum_insured DECIMAL(18,2),
  policy_status STRING,
  effective_date DATE,
  expire_date DATE,
  uw_result STRING,
  issue_datetime TIMESTAMP,
  etl_time TIMESTAMP
) PARTITIONED BY (dt STRING)
STORED AS PARQUET;
```

5. Storage Selection

Data Type	Engine	Use Case
OLTP	MySQL 8.0 / PolarDB	Core business
Cache	Redis Cluster	Hot data, sessions
Search	Elasticsearch	Policy search, logs
Time-series	Prometheus / VictoriaMetrics	Metrics
Batch	Spark + Hive/Iceberg	ETL, DWH
Real-time	Flink + ClickHouse	Real-time dashboards
Object	MinIO / OSS / S3	Images, files
Message	RocketMQ / Kafka	Events, streaming

6. Data Pipeline Standards

6.1 Real-time Pipeline

MySQL(Binlog) → Canal → Kafka → Flink → ClickHouse / Redis

- At-least-once delivery, idempotent consumers

6.2 Offline Pipeline

MySQL(Sqoop/DataX) → HDFS → Hive(ODS) → Spark(DWD/DWS) → ClickHouse(ADS)

7. Data Governance

Area	Requirement	Tool
Data Quality	Automated monitoring for null rates, uniqueness, format	Great Expectations / Deequ
Data Lineage	Field-level lineage tracking	Atlas / DataHub
Metadata	Table structures, field comments, change history	Custom metadata platform
Data Security	Auto-discovery + masking + encryption	Data security platform
Lifecycle	Hot/cold tiering, auto-cleanup	Lifecycle management

8. Regulatory Reporting

- Regulatory data processed directly from ODS/DWD, bypassing ADS
- All reported data retained for at least 7 years
- Pre-submission reconciliation with source systems
- Reporting interfaces are additive only, historical versions must remain compatible